南昌航空大學

毕业设计（论文）

题　　目：面向航空旅客的智能美食推荐系统的设计与实现

学　　院：信息工程学院

专业名称：计算机科学与技术

班级学号：22045309

学生姓名：胡屿科

指导教师：

二零二五年六月

南昌航空大學

毕业设计（论文）任务书

I、毕业设计(论文)题目：

面向航空旅客的智能美食推荐系统的设计与实现

II. 毕业设计（论文）使用的原始资料（数据）及设计技术要求：

原始资料：

1. 毕业设计任务书和相关文献资料
2. 网上公开的航空餐饮数据库·用户饮食偏好数据集、以及推荐算法登相关技术文档。

设计要求：

1. 设计并实现面向航空旅客的智能美食推荐系统，实现基于旅客偏好、航班信息和餐饮数据的个性化美食推荐。
2. 系统应具备实时性、稳定性与可用性，推荐算法需达到较好的准确率与满意度，界面直观、操作简便。
3. 用户端：美食推荐列表查看与筛选、个人饮食偏好管理、历史推荐记录查询、查看航班餐饮公告。
4. 管理端：美食推荐数据看板、餐饮资源管理、航班与公告信息发布、旅客偏好数据分析。

III、毕业设计(论文)工作内容及完成时间：

1. 第 1 周-第 3 周：查阅相关资料，研究推荐系统相关技术及航空服务场景需求，明确功能模块和技术方案，完成开题报告和论文翻译。
2. 第 4 周：完成系统总体架构设计、数据库设计和功能模块划分。
3. 第 5 周-第 7 周：实现智能美食推荐核心算法，包括数据预处理、特征工程与模型训练。
4. 第 8 周-第 11 周：完成业务逻辑开发和数据库实现，整合前后端功能。
5. 第 12 周-第 15 周：整合推荐算法模块，进行系统功能测试、性能优化和用户体验改进，完成毕业设计论文的撰写。
6. 第 16 周：修改毕业论文、论文答辩、整理毕业设计相关文档。

Ⅳ、主要参考资料：

[1] 孟佩,党小娟,刘冬,等.基于深度学习和多领域融合的个性化美食与旅游推荐系统[J].电脑知识与技术,2025,21(22):70-73.

[2] 尤天逸,赵郑博,江楠,等.基于改进协同过滤算法的个性化美食推荐APP开发[J].信息与电脑(理论版),2024,36(22):162-164.

[3] 刘建业.基于注意力与残差机制的深度协同推荐算法及其应用[D].北京交通大学,2024.

[4] 黄桂吉,刘盾,叶晓庆.一种基于粒计算的美食推荐方法[J].昆明理工大学学报(自然科学版),2023,48(03):93-104.

[5] 刘锐,徐鲁辉.基于Flink的智能餐饮推荐系统研究[J].信息技术与信息化,2022,(10):204-207

[6] 邓涵兮,陈志华.基于网络评论的美食推荐系统[J].中国传媒科技,2022,(03):39-41.

[7] 周蕾,李强.基于LBS应用的淮安美食推荐类系统的研究[J].食品安全导刊,2021,(21):172-173.

[8] 王佳安,王可心,李直旭.美食精准搜索与智能推荐平台的设计与实现[J].福建电脑,2019,35(08):27-30.

[9] Tellechea Y ,Arrojo M ,Cejudo A , et al.Population-Level Analysis of Personalized Food Recommendation Using Reinforcement Learning[J].Foods,2025,14(21):3770-3770.

[10] Zhao H ,Chen H ,Wang J , et al.Cross-View Heterogeneous Graph Contrastive Learning Method for Healthy Food Recommendation[J].Computation,2025,13(8):197-197.

信息工程 **学院** 计算机科学与技术 **专业类** 220453 **班**

学生（签名）：

日期： 自 2026 年 01 月 06 日至 2025 年 06 月 01 日

指导教师（签名）：

助理指导教师(并指出所负责的部分)：

计算机应用技术 **系（室）主任（签名）：**

附注:任务书应该附在已完成的毕业设计说明书首页。

南昌航空大學

学士学位论文原创性声明

本人声明，所呈交的学位论文是本人在指导教师的指导下独立完成的研究成果。除了文中特别加以标注引用的内容外，本论文不包含法律意义上已属于他人的任何形式的研究成果，也不包含本人已用于其他学位申请的论文或成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式表明。本人完全意识到本声明的法律后果由本人承担。

作者签名：　　日期：

导师签名：　　日期：

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权南昌航空大学可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

作者签名：　　日期：

导师签名：　　日期：

摘　要

随着民航业快速复苏，机上餐食服务成为影响旅客满意度的关键环节。传统标准化配餐模式难以满足旅客口味偏好与饮食需求的个性化差异，导致餐食浪费与旅客体验不佳并存。基于此背景，本文设计并实现了一个面向航空旅客的智能美食推荐系统，形成"偏好采集—智能推荐—预选确认—航后评分"的完整闭环。

本文提出了一种自适应混合推荐算法 Fused-PMFUP-PRMIDM-AMMBC，由三个子模型构成：PMFUP 基于用户画像进行餐食类型与口味标签的内容匹配；PRMIDM 通过对比近 30 天与近 180 天行为信号，捕捉旅客口味短期漂移趋势；AMMBC 采用基于用户的协同过滤，引入收缩因子、流行度去偏与置信度加权策略。三模型权重根据用户行为信号强度自适应调整，冷启动时 PMFUP 占主导，随信号积累协同过滤与漂移模型逐步提升权重。系统引入舱位等级递进规则与前序餐食惩罚机制，以 Spring Boot 3 + Vue3 + UniApp 技术栈实现管理端、小程序端与后端服务三端架构。

在 90 人模拟数据集上的测试表明，算法覆盖冷启动、多餐食连续推荐、舱位差异化等典型场景，能够生成带可解释性输出的个性化推荐列表，各功能模块运行稳定，满足毕业设计的演示与答辩要求。

关键词：推荐系统；航空餐食；混合推荐算法；协同过滤；兴趣漂移；Spring Boot

Abstract

With the rapid recovery of civil aviation, in-flight meal service has become key to passenger satisfaction. Traditional standardized meal distribution cannot accommodate individual differences in taste and dietary needs, leading to both food waste and poor experience. This paper designs an intelligent food recommendation system for aviation passengers, forming a closed loop of "preference collection—intelligent recommendation—pre-selection confirmation—post-flight rating."

A fused adaptive hybrid recommendation algorithm, Fused-PMFUP-PRMIDM-AMMBC, is proposed with three sub-models: PMFUP performs content-based matching on meal types and flavor tags from user profiles; PRMIDM captures short-term flavor drift by comparing behavioral signals within a 30-day recent window against a 180-day historical window; AMMBC employs user-based collaborative filtering with shrinkage, popularity debiasing, and confidence weighting. Fusion weights adapt to signal strength, with PMFUP dominating in cold-start and collaborative/drift models gaining weight as signals accumulate. A cabin-class tiered visibility rule and sequential meal penalty are introduced. The system uses Spring Boot 3 + Vue3 + UniApp across admin, mini-program, and backend.

Tests on a 90-user simulated dataset covering cold-start, multi-meal recommendation, and cabin-tier scenarios demonstrate personalized recommendation lists with explainable outputs. All modules operate stably, meeting graduation defense requirements.

Keywords: Recommendation System; Aviation Catering; Hybrid Recommendation; Collaborative Filtering; Interest Drift; Spring Boot

目录

1.1 研究背景及意义 11
1.2 国内外发展现状与发展趋势 12
1.2.1 国内研究概况 12
1.2.2 国外研究概况 12
1.2.3 航空餐食推荐发展历程 13
1.2.4 航空餐食推荐发展趋势 13
1.3 研究内容及结构 15
2.1 推荐系统概述 16
2.1.1 推荐系统基本概念 16
2.1.2 推荐系统主要方法分类 16
2.2 协同过滤 17
2.2.1 基于用户的协同过滤 17
2.2.2 余弦相似度 ..... **错误！未定义书签。**
2.2.3 收缩因子修正 18
2.3 基于内容的推荐 19
2.4 混合推荐方法 19
2.5 推荐系统关键问题 20
2.5.1 冷启动问题 20
2.5.2 用户兴趣漂移 20
2.5.3 流行度偏差 21
2.6 推荐系统评价指标 21
2.7 系统开发技术选型 22
2.7.1 后端技术：Spring Boot 3 与 MyBatis 22
2.7.2 前端技术：Vue3、Element Plus 与 UniApp 23
2.7.3 数据存储：MySQL 与 Redis 23
2.8 本章小结 24
3.1 算法总体设计 24
3.2 行为数据预处理 25
3.2.1 数据源与行为信号 25
3.2.2 交互上下文构建 25
3.3 子模型一：PMFUP——基于用户画像的偏好匹配 26
3.4 子模型二：PRMIDM——偏好漂移与兴趣迁移模型 27
3.5 子模型三：AMMBC——自适应协同过滤 28
3.6 自适应权重融合与多餐约束 29
3.6.1 融合权重自适应调整 30
3.6.2 多餐前序惩罚与解释性输出 31
3.7 算法总体流程 31
5.1 测试环境 38
5.2 测试数据构建 38
5.3 推荐算法效果分析 38
5.3.1 融合权重自适应验证 39
5.3.2 三子模型评分分解 39
5.3.3 多餐前序惩罚效果 40
5.3.4 舱位差异化推荐验证 41
5.4 系统功能验证 41

5.4.1 管理端功能验证 41
5.4.2 小程序端功能验证 42
5.5 本章小结 42
6.1 研究工作总结 44
6.2 研究不足与展望 44

第 1 章引言

1.1 研究背景及意义

随着全球民用航空业的持续复苏，机上餐食服务已从传统"配餐保障"转变为影响旅客满意度的关键服务触点。2024 年我国民航完成旅客运输量 7.3 亿人次，全国航空配餐企业年配餐量近 7 亿份。在如此庞大的服务规模下，机上餐食的品质与个性化程度直接影响航空公司的品牌竞争力与运营效率。

然而，传统的机上配餐模式长期沿用"标准化、批量供应"策略，旅客在值机时面对有限的两到三种餐食选项，无法根据个人口味偏好、饮食禁忌或健康需求进行自主选择。这一模式面临三个核心矛盾：其一，旅客饮食需求的多样化与餐食供给的标准化之间的矛盾——不同旅客在口味、餐食类型和健康偏好上存在显著差异，标准化配餐难以覆盖长尾需求；其二，航空公司的成本控制与餐食浪费之间的矛盾——全球航空业每年产生约 360 万吨客舱和餐食垃圾，其中 18%至 20%为未食用餐食，过度装载导致的浪费直接转化为燃油消耗与碳排放的增加；其三，旅客期望的实时个性化服务与现有配餐流程信息滞后之间的矛盾——旅客偏好数据在订票、值机、登机各环节碎片化分布，缺乏统一的采集与智能推荐机制。

从理论层面看，个性化推荐系统在电子商务、社交媒体等领域已取得丰硕成果，协同过滤、基于内容的推荐与混合推荐等方法日趋成熟[1][2]。然而，将通用推荐技术迁移到航空餐食这一垂直场景时，面临一系列特有约束：航班餐食供给受航线、舱位与库存的多维约束，推荐结果必须在实际可供应范围内；旅客餐食偏好可能随季节、航线等因素发生短期漂移，静态偏好模型难以捕捉动态变化[3]；新旅客首次乘机缺乏历史行为数据，构成典型的冷启动问题[4]；多餐食航班上需考虑不同餐次的推荐多样性，避免重复推荐[5]。上述约束使得航空餐食推荐成为兼具理论价值与应用价值的研究问题。

从应用层面看，构建一个面向航空旅客的智能美食推荐系统具有多重现实意义：对旅客而言，个性化推荐能提升服务体验，满足多元化饮食需求；对航空公司而言，基于偏好预测的精准配餐能减少餐食浪费、降低运营成本；对行业而言，该系统可为铁路、邮轮等其他交通领域的餐饮服务数字化提供参考范式。

基于此背景，本文设计并实现了一个面向航空旅客的智能美食推荐系统，提出一种融合用户画像匹配、兴趣漂移建模与协同过滤的自适应混合推荐算法，并基于 Spring Boot + Vue.js + UniApp 技术栈构建了管理端、小程序端与后端服务三端系统，形成"偏好采集—智能推荐—预选确认—航后评分"的完整闭环。

1.2 国内外发展现状与发展趋势

1.2.1 国内研究概况

国内推荐系统的研究起步虽晚于国外，但近年来在算法改进与行业应用方面均取得显著进展。在协同过滤与混合推荐方面，余江龙等人从多维度综述了个性化推荐系统的研究方法[1]；王素芳等人将协同过滤与内容特征相结合，缓解了数据稀疏场景下的精度下降问题[2]。在兴趣漂移与动态偏好方面，夏翔等人在相似度计算中引入时间权重与兴趣迁移模型[3]；钱聪等人融合时间衰减函数与评论语义相似度，捕捉用户兴趣的渐变性变化[6]；刘超等人通过提取长短期偏好并自适应聚合来处理序列推荐中的偏好漂移[5]。在冷启动问题上，毛骞等人按数据驱动与方法驱动两类进行了系统综述[4]。在协同过滤相似度优化方面，刘昊东等人在余弦相似度中引入 Jaccard 函数与热门度修正因子[7]，并结合置信度加权策略降低流行偏差对推荐结果的影响。

在餐饮推荐领域，陈南平从用户就餐评论中提取情境信息构建推荐模型[9]；孟佩等人结合协同过滤与多领域数据融合应用于美食推荐[10]；李凯提出了一种融合项目特征与用户偏好的美食推荐方法[11]，为本文提供了直接参考。在航空餐食信息化建设方面，北京航食于 2024 年上线了旅客餐食预订管理系统，贯通 19 套系统实现了全链条数字化管理。厦门航空率先推出经济舱免费选餐服务，东航申请了航班配餐量智能预测专利。但将个性化推荐算法深度嵌入航空餐食场景、形成完整闭环系统的工程实践仍较为缺乏。

1.2.2 国外研究概况

国外推荐系统研究起步较早，理论体系更为成熟。在基础理论方面，Koren 等人提出的矩阵分解技术[16]通过学习用户与物品的隐语义表示，显著提升了推荐精度与可扩展性，为协同过滤奠定了数学基础。在食品推荐领域，Bondevik 等人对食品推荐系统进行了系统性综述[17]，指出多数系统依赖显式偏好与物品属性特征进行匹配，而将行为反馈融入推荐闭环的研究仍较为薄弱。

在航空餐食智能化方面，Dwarampudi（2025）提出了基于 AI 的机上餐食个性化框架[18]，比较了协同过滤、旅客细分、强化学习和神经网络四条技术路径。在产业实践方面，汉莎航空推出"餐盘追踪器"系统，通过 AI 视觉识别回收餐食的剩余状态以优化配餐量；荷兰皇家航空的 TRAYS 模型据报可减少 63%的餐食浪费；空客公司于 2025 年推出"Smart Catering"解决方案，通过机载摄像头与云端偏好分析实现智能配餐。总体来看，国外研究更注重端到端的数据闭环，但仍面临真实数据稀缺、个性化与成本平衡等挑战。

1.2.3 航空餐食推荐发展历程

航空餐食服务模式大致经历了四个发展阶段。第一阶段（20 世纪 50 年代至 90 年代）为标准化配餐阶段，机上餐食作为基础飞行保障服务批量供应，不考虑个性化需求。

第二阶段（20 世纪 90 年代末至 2010 年代）为特殊餐食预申请阶段，航空公司开始在购票环节提供清真餐、素食餐等特殊选项，旅客需提前 24 至 48 小时申请，但普通旅客仍仅有标准餐可选。

第三阶段（2010 年代至 2020 年代初）为数字化预选阶段，航空公司通过 App 或网站提供餐食预选服务，但推荐逻辑以基于规则的简单排序为主，尚未实现个性化推荐。

第四阶段（2020 年至今）为智能推荐探索阶段，航空公司开始利用机器学习算法分析旅客偏好与行为信号，AI 视觉识别技术被引入餐盘回收环节，形成从"推荐→消费→反馈"的数据闭环，这一阶段的核心特征是推荐系统、需求预测与数据闭环的初步融合，也是本文工作的技术背景。

1.2.4 航空餐食推荐发展趋势

综合国内外研究现状与航空餐食服务模式的演进，该领域呈现以下发展趋势。（1）从简单规则到混合算法。早期主要基于销量排序等简单规则，未来将向多模型融合的混合推荐算法演进，协同过滤、内容匹配与行为反馈的互补融合将显著提升推荐精度[2]。（2）从静态画像到动态偏好。传统的静态偏好采集正在被时间衰减函数与兴趣漂移检

测技术所替代[3][6]，通过对比近期行为与长期历史实现自适应偏好更新。（3）从单向推荐到闭环反馈。在推荐曝光、点击浏览、预选确认、航后评分等环节埋设行为追踪点，构建从"曝光→交互→选择→评价"的数据闭环，使推荐算法持续迭代优化[11]。（4）从单次推荐到序列推荐。多餐食航班上需考虑同一旅客前序餐次的选择，避免重复推荐，序列推荐方法将多餐推荐建模为时序决策问题[5]。（5）从同质化到多层次差异化。引入舱位等级递进的餐食可见规则与差异化推荐策略，实现头等舱、商务舱与经济舱旅客的精准服务。（6）从数据孤岛到全链路数字化。当前旅客偏好数据在订票、值机、配餐、客舱等系统间分散存储，构建统一的偏好数据平台是全链路数字化的关键方向。

1.3 研究内容及结构

本文旨在设计并实现一个面向航空旅客的智能美食推荐系统，提出一种融合用户画像匹配、兴趣漂移建模与协同过滤的自适应混合推荐算法 Fused-PMFUP-PRMIDM-AMMBC，将航班供给约束、旅客偏好与行为反馈融入推荐决策。各章节安排如下：

第一章为引言，阐述研究背景、国内外研究现状及发展趋势。

第二章为相关基础知识，介绍推荐系统基础理论与系统技术选型。

第三章为推荐算法原理与实现，阐述三子模型的设计思想、计算公式与代码实现。

第四章为系统设计与实现，按照软件工程方法论从需求分析、架构设计到各模块实现进行系统阐述。

第五章为系统测试与结果分析，展示推荐算法在不同场景下的推荐效果。

第六章为结论与展望，总结研究工作与主要成果，分析不足并展望改进方向。

第 2 章 相关基础知识

2.1 推荐系统概述

2.1.1 推荐系统基本概念

推荐系统（Recommender System）是一种用于解决信息过载问题的信息过滤技术，其核心任务是根据用户的历史行为、个人偏好以及物品的属性特征，从海量候选物品中筛选出用户可能感兴趣的物品子集，并以个性化列表的形式呈现给用户[1]。与搜索引擎不同，推荐系统不需要用户主动输入明确的查询关键词，而是通过分析用户的隐式行为（如浏览、点击、选择）和显式反馈（如评分、点赞）自动推断用户的兴趣偏好。

一个典型的推荐系统通常由三个核心模块构成：用户建模模块负责收集和表示用户偏好；推荐算法模块根据用户模型与物品模型计算预测评分或排序；结果输出模块将推荐结果以列表形式展示并收集用户反馈。推荐问题的数学形式可以描述为：设用户集合为 U，物品集合为 I，推荐系统的目标是学习一个预测函数 $f: U \times I \to \mathbb{R}$，使得 $f(u,i)$ 能够表示用户 u 对物品 i 的预测偏好程度，系统据此对候选物品进行排序，选择得分最高的 K 个物品推荐给用户。

2.1.2 推荐系统主要方法分类

根据推荐策略的不同，主流的推荐系统方法可分为以下三类[1]：

（1）协同过滤推荐（Collaborative Filtering，CF）。协同过滤基于"物以类聚，人以群分"的基本假设，利用用户群体的行为数据进行推荐。其核心思想是找到与目标用户行为相似的其他用户，根据相似用户的偏好来预测目标用户的偏好。协同过滤不依赖于物品的内容特征，仅利用用户-物品交互矩阵（如评分矩阵或行为矩阵），因此具有良好的领域适用性。

（2）基于内容的推荐（Content-Based Recommendation）。基于内容的推荐从物品自身的属性特征出发，通过分析用户过去喜欢的物品的内容特征，构建用户兴趣画像，然后将待推荐物品的特征与用户画像进行匹配。这类方法不依赖其他用户的数据，因此不存在物品冷启动问题，但要求物品具有可解析的内容特征。

（3）混合推荐（Hybrid Recommendation）。混合推荐将多种推荐方法以某种策略进行组合，以弥补单一方法的不足，发挥各种方法的优势。常见的混合策略包括加权融合、切换式混合、特征组合和分层混合等[2]。本文提出的自适应混合推荐算法即属于加权融合策略的范畴，通过根据用户行为信号强度动态调整各子模型的权重，实现协同过滤、内容匹配与行为反馈的优势互补。

2.2 协同过滤

2.2.1 基于用户的协同过滤

基于用户的协同过滤（User-Based Collaborative Filtering）是协同过滤中最经典的方法之一[7]。其基本思想分为两个步骤：首先计算目标用户与其他所有用户之间的相似度，找到与目标用户兴趣最相近的 K 个邻居用户；然后根据邻居用户对目标物品的评分或行为，加权预测目标用户对该物品的偏好程度。

设用户 u 与用户 v 的行为向量分别为$\vec{u}$和$\vec{v}$，用户相似度为 $\mathrm{sim}(u,v)$，则用户 u 对物品 i 的预测得分 $\mathrm{p}(u,i)$ 可通过以下公式计算：

$$\mathrm{p}(u,i)=\frac{\sum_{v\in N(u)} s\,im(u,v)\cdot r(v,i)}{\sum_{v\in N(u)} s\,im(u,v)}$$

其中 $\mathrm{N}(u)$ 表示用户 u 的邻居用户集合（只考虑那些对物品 i 有过交互行为的邻居），$\mathrm{r}(v,i)$ 表示用户 v 对物品 i 的行为得分。基于用户的协同过滤的优点在于能够发现用户潜在的兴趣领域，推荐结果具有一定的惊喜度（Serendipity）；其缺点在于当用户数量庞大时，相似度计算的复杂度较高，且新用户面临冷启动问题。

2.2.2 余弦相似度

余弦相似度（Cosine Similarity）是协同过滤中计算用户间相似度最常用的方法之一[8]。它将每个用户的行为向量视为高维空间中的一个向量，通过计算两个向量之间夹角的余弦值来衡量用户的相似程度。对于两个用户 u 和 v，其余弦相似度的计算公式为：

$$\cos(u,v)=\frac{\vec{u}\cdot\vec{v}}{\|\vec{u}\|\times\|\vec{v}\|}=\frac{\sum_{i\in I_{uv}} r_{u,i}\times r_{v,i}}{\sqrt{\sum_{i\in I_u} r_{u,i}^2}\times\sqrt{\sum_{i\in I_v} r_{v,i}^2}}$$

其中 I_u 和 I_v 分别表示用户 u 和用户 v 有过行为记录的物品集合，I_{uv} 表示两个用户共同交互过的物品集合，$r_{u,i}$ 表示用户 u 对物品 i 的行为得分。余弦相似度的取值范围为 $[0,1]$（在非负向量空间中），值越大表示两个用户的偏好越相似。当两个用户没有任何共同交互物品时，余弦相似度为 0。

余弦相似度的一个优势在于它对向量的模长不敏感，仅关注向量的方向（即偏好的相对分布），因此能够在一定程度上缓解不同用户评分尺度差异带来的影响。然而，当两个用户的共同交互物品数量很少时，余弦相似度可能会高估用户的相似性——例如，两个用户仅在一种物品上有共同交互且得分恰好相同，余弦相似度将等于 1，但这很可能是一种偶然现象。

2.2.3 收缩因子修正

针对小重叠度下余弦相似度可能产生假性高相似的问题，本文系统引入了收缩因子（Shrinkage）对用户相似度进行修正[12]。收缩因子的基本思想是：当两个用户的共同交互物品数量较少时，对相似度施加惩罚，使其向零收缩；随着共同交互物品数量的增加，修正后的相似度逐步逼近原始值。修正公式为：

$$\text{adjusted_sim}(u, v) = \text{sim}(u, v) \times \frac{overlap}{overlap + \lambda}$$

其中 overlap 表示两个用户的共同交互物品数量，λ 为收缩系数（本文系统中取 $\lambda = 3.0$）。当 $\text{overlap} = 1$ 时，修正系数为 $1/(1+3) = 0.25$，原始相似度被大幅削弱；当 $\text{overlap} = 10$ 时，修正系数为 $10/(10+3) \approx 0.77$，相似度被适度保留；当 overlap 趋于无穷大时，修正系数趋于 1，修正后的相似度等价于原始相似度。该机制使得只有在拥有足够共同交互证据支撑的情况下，两个用户才会被认定为高度相似，从而提升了协同过滤推荐的可靠性。

2.3 基于内容的推荐

基于内容的推荐方法从物品自身的属性特征出发，通过构建用户兴趣画像并与物品特征进行匹配来生成推荐[13]。与协同过滤不同，基于内容的推荐不依赖于其他用户的行为数据，而是利用物品的内容属性（如文本描述、类别标签、属性特征等）和用户的历史偏好记录来进行推荐。

基于内容的推荐通常包含三个步骤。第一步是物品特征提取，即将物品的属性信息转化为结构化的特征向量。在本系统中，每一道菜品具有两种关键的内容特征：餐食类型（mealType），取值为儿童餐、标准餐、清真餐、素食四类之一；口味标签集合（flavorTags），包括清淡、咸香、微辣、甜口、低脂、高蛋白共六种口味。第二步是用户画像构建，即根据用户的历史交互记录或显式偏好设置，生成用户的兴趣特征向量。在本系统中，用户的兴趣画像来源于两个方面：用户在偏好采集页面主动设定的餐食类型偏好和口味偏好，以及用户历史行为日志中累积的菜品交互得分。第三步是匹配度计算，即将候选物品的特征向量与用户兴趣画像进行匹配，计算用户对物品的兴趣度得分。在本系统中，匹配度的计算考虑了餐食类型的匹配奖励（单偏好 +0.45 / 多偏好 +0.38）和口味标签的重叠度奖励（$\min(0.35, 0.15 + matchedCount \times 0.10)$）。

基于内容的推荐的主要优势在于：不依赖其他用户数据，因此不存在物品冷启动问题；推荐结果具有较好的可解释性，可以明确告知用户推荐理由（如"符合您的清淡口味偏好"）；能够有效推荐新上架的物品或长尾物品。其主要局限在于：受限于物品特征的丰富程度，难以挖掘用户的潜在兴趣；推荐结果可能缺乏多样性，容易形成"信息茧房"效应。

2.4 混合推荐方法

混合推荐方法通过将多种推荐策略以一定方式进行组合，取长补短，以克服单一推荐方法的局限性[2]。常见的混合策略包括以下四种：

（1）加权融合（Weighted Fusion）。为每种推荐方法赋予一个权重系数，最终推荐得分由各方法得分的加权和决定：$S_{final} = w_1 S_1 + w_2 S_2 + \cdots + w_n S_n$。加权融合的关键在于权重的确定方式——可以是固定权重（如基于经验设定），也可以是自适应权重（如根据用户行为特征动态调整）。

（2）切换式混合（Switching Hybrid）。根据当前场景的条件（如用户是否有足够的历史行为数据），在多种推荐策略之间进行切换。例如，当用户行为信号充足时采用协同过滤，而当用户为新用户（冷启动场景）时切换为基于内容的推荐。

（3）特征组合（Feature Combination）。将多种推荐策略产生的中间特征拼接或组合，作为统一模型的输入特征。例如，将协同过滤产生的用户相似度特征与基于内容推荐的物品匹配度特征拼接后输入到元学习器中进行二次学习。

（4）分层混合（Meta-Level Hybrid）。使用一种推荐方法产生的模型或中间结果作为另一种推荐方法的输入。例如，先用基于内容的方法构建用户画像，再将用户画像作为协同过滤中相似度计算的特征维度。

本文系统采用的自适应加权融合策略同时具备加权融合与切换式混合的特征：一方面，三个子模型（内容匹配 PMFUP、兴趣漂移 PRMIDM、协同过滤 AMMBC）的得分通过加权和进行融合；另一方面，融合权重根据用户行为信号强度自适应调整——当信号稀疏时内容匹配子模型占主导，随信号积累协同过滤子模型的权重逐步提升，实现了从冷启动到个性化推荐的平滑过渡。

2.5 推荐系统关键问题

2.5.1 冷启动问题

冷启动问题（Cold Start Problem）是推荐系统面临的核心挑战之一[4]。当新用户首次使用推荐系统时，系统缺乏该用户的历史行为数据，无法通过协同过滤或行为分析来推断其偏好，导致推荐质量显著下降。冷启动问题通常分为用户冷启动（新用户无历史记录）和物品冷启动（新物品无交互记录）两类，在航空餐食推荐场景中，用户冷启动是主要挑战。

解决用户冷启动的常见策略包括：（1）利用用户注册时填写的显式偏好信息（如口味偏好、饮食禁忌）进行基于内容的推荐；（2）利用用户的人口统计学特征（如年龄、性别）与已有用户进行粗粒度匹配；（3）推荐全局热门物品作为初始推荐，随着用户行为数据的积累逐步过渡到个性化推荐；（4）采用混合推荐策略，在冷启动阶段提高内容匹配的权重，随着行为信号的增加逐步提升协同过滤的权重。本文系统综合采用了策略（1）和策略（4）：新用户首次使用时引导其完成偏好采集，并在偏好未完成时仍允许跳过直接体验推荐；推荐算法在行为信号稀疏时自动提升内容匹配子模型（PMFUP）的融合权重，随着用户交互数据的积累逐步降低其权重。

2.5.2 用户兴趣漂移

用户兴趣漂移（Interest Drift）是指用户的偏好和兴趣随时间发生变化的自然现象[6]。在推荐系统中，如果模型仅基于用户的长期历史行为构建静态偏好画像，而忽略了兴趣的动态变化，将导致推荐结果与用户当前的偏好产生偏差，推荐时效性和精准度下降。

处理兴趣漂移的常用方法包括：（1）时间衰减函数，即为用户的历史行为赋予随时间递减的权重，使得近期行为的贡献大于远期行为；（2）长短期偏好分离，即分别维护用户的长期稳定偏好和短期动态偏好两个模型，并在推荐时进行动态融合；（3）滑动时间窗口，即仅考虑最近一段时间内的用户行为进行偏好建模。本文系统综合采用了时间衰减函数与长短期偏好分离两种策略：行为信号通过指数衰减函数 $e^{-0.018\times days}$ 进行时间加权（半衰期约 38.5 天）；同时分别维护近 30 天近期窗口和近 180 天历史窗口的两套口味偏好信号，通过对比近期偏好与历史偏好来检测口味漂移方向，对发生正向漂移的口味所对应的菜品给予得分提升。

2.5.3 流行度偏差

流行度偏差（Popularity Bias）是指推荐系统倾向于推荐已被大量用户选择的热门物品，而忽视了长尾物品和新物品的现象[3]。这种偏差在协同过滤中尤为明显——热门物品由于被大量用户交互过，在相似度计算和评分预测中占据优势，形成"富者愈富"的马太效应。流行度偏差不仅降低了推荐结果的多样性和个性化程度，还可能导致新物品难以获得曝光机会。

缓解流行度偏差的常用方法包括：（1）流行度惩罚因子，即在推荐得分中引入与物品流行度成反比的惩罚项，降低热门物品的得分优势；（2）反事实推理，即通过因果推断方法消除流行度对推荐结果的混杂影响；（3）去偏正则化，即在模型训练目标函数中加入流行度相关的正则化项。本文系统在协同过滤子模型（AMMBC）中引入了流行度去偏机制：对于每个候选菜品，根据其在推荐日志中的累计曝光次数计算流行度惩罚系数 $1.0/\left(1.0+\frac{\ln(1+exposureCount)}{6.0}\right)$，曝光次数越多，惩罚系数越低，从而在协同过滤的得分中削弱热门菜品的先发优势。同时，协同过滤子模型还引入了置信度加权策略：$clamp(simTotal/\max(1.0, neighborCount\times 0.45))$，在邻居用户数量不足或相似度总量较低时降低协同过滤得分的可信度，进一步提升推荐的稳定性。

2.6 推荐系统评价指标

推荐系统的评价指标用于量化评估推荐结果的质量。根据推荐任务的不同视角，评价指标通常分为预测精度指标、分类指标和排序指标三类[19]。

准确率（Accuracy）是最基础的分类指标，表示模型预测正确的样本数占总样本数的比例：

$$Accuracy=\frac{TP+TN}{TP+TN+FP+FN}$$

其中 TP（True Positive）表示正确预测为正样本的数量，TN（True Negative）表示正确预测为负样本的数量，FP（False Positive）表示错误预测为正样本的数量，FN（False Negative）表示错误预测为负样本的数量。在推荐场景中，正样本通常为用户实际选择或评分的物品。

精确率（Precision）衡量推荐列表中真正为用户感兴趣的物品所占的比例，反映了推荐结果的"精准度"：

$$Precision = \frac{TP}{TP + FP}$$

召回率（Recall）衡量用户真正感兴趣的所有物品中有多大比例被推荐系统成功找出，反映了推荐结果的"查全率"：

$$Recall = \frac{TP}{TP + FN}$$

F1 分数（F1-Score）是精确率与召回率的调和平均数，用于在两个指标之间取得平衡：

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

AUC（Area Under the ROC Curve）是评估推荐模型排序质量的重要指标。ROC 曲线以假正率（FPR）为横轴、真正率（TPR）为纵轴，通过改变分类阈值绘制，AUC 值即为 ROC 曲线下的面积，取值范围为[0.5,1]。AUC 值越大，表示模型区分正负样本的能力越强：$AUC = 0.5$ 表示模型预测能力等同于随机猜测，$AUC = 1$ 表示模型能完美区分所有正负样本。

2.7 系统开发技术选型

2.7.1 后端技术：Spring Boot 3 与 MyBatis

本系统后端采用 Spring Boot 3.2.5 作为核心开发框架，基于 JDK 17 运行环境。Spring Boot 是 Spring 生态体系的快速开发框架，通过自动配置（Auto-Configuration）和内嵌服务器（Embedded Server）大幅简化了 Spring 应用的搭建与部署流程。其主要优势包括：起步依赖（Starter）机制将相关依赖打包为单一引用，减少了依赖管理的复杂性；内嵌 Tomcat 服务器使应用可以以独立 JAR 包形式运行，无需外部 Servlet 容器；与 Spring Security、Spring Data 等 Spring 生态组件的无缝集成能力。

数据持久层采用 MyBatis 3.0.3 作为 ORM 框架。MyBatis 是一种半自动化的持久层框架，通过 XML 映射文件或注解将 Java 对象与 SQL 语句关联，开发者可以直接编写和优化 SQL 语句，在保持 SQL 灵活性的同时避免了 JDBC 的繁琐编码。本系统通过 application.yml 配置文件指定了 Mapper 文件的扫描路径（`classpath:mapper/*.xml`）、实体类别名包（`fun.hykgraph.entity`）以及驼峰命名自动映射（`map-underscore-to-camel-case: true`），共包含 11 个 Mapper XML 映射文件和对应的 Java Mapper 接口，覆盖用户、航班、菜品、偏好、推荐日志、预选订单与评分等核心数据表。

2.7.2 前端技术：Vue3、Element Plus 与 UniApp

管理端前端采用 Vue 3.4.21 作为核心框架，配合 TypeScript 5.4.0 进行类型安全的开发，使用 Vite 5.2.8 作为构建工具。Vue 3 的组合式 API（Composition API）允许将组件逻辑按功能进行模块化组织，提升了代码的可复用性和可维护性。UI 组件库采用 Element Plus 2.7.1，提供表格、表单、对话框、导航菜单等丰富的企业级组件。数据可视化采用 ECharts 5.5.0，用于管理端数据看板的图表展示。状态管理采用 Pinia 2.1.7，配合持久化插件（pinia-plugin-persistedstate 3.2.1）将关键状态自动同步至浏览器本地存储。HTTP 客户端采用 Axios 1.6.8，用于向后端发送 RESTful API 请求。

小程序端采用 UniApp 3.0.0 跨端开发框架，基于 Vue 3.3.11 构建，编译目标为微信小程序（mp-weixin）。UniApp 通过统一的代码基输出到多端平台，降低了多端维护成本。小程序端集成了微信一键登录功能，通过调用 `wx.login` 获取临时凭证后向后端请求 OpenID 并生成 JWT 令牌，后续请求通过 HTTP Header 携带令牌完成身份认证。

2.7.3 数据存储：MySQL 与 Redis

本系统采用 MySQL 作为主数据库，数据库名为 `aviation_food_recommend`，运行于本地 3306 端口。MySQL 是一种成熟的关系型数据库管理系统，支持 ACID 事务特性，适合存储具有强关联关系的结构化数据。本系统的核心数据表包括用户表（user）、航班信息表（flight_info）、菜品表（dish）、用户偏好表（user_preference）、推荐日志表（recommendation_log）、餐食预选表（meal_selection）、评分任务表（flight_service_rating）和公告表（flight_announcement）等，表之间通过外键和索引建立关联约束。

本系统采用 Redis 作为缓存数据库，运行于本地 6379 端口（db 1）。Redis 是一种基于内存的高性能键值存储数据库，支持字符串、哈希、列表、集合和有序集合等多种数据结构。在本系统中，Redis 主要用于缓存热点数据（如航班列表、菜品信息），以降低数据库查询压力，提升接口响应速度。

2.8 本章小结

本章围绕航空餐食智能推荐系统所涉及的基础理论与关键技术进行了系统介绍。首先从推荐系统的概念与分类入手，阐述了三类主要推荐方法的原理与特点；其次深入介绍了协同过滤推荐的核心机制——基于用户的协同过滤、余弦相似度计算以及收缩因子修正策略；然后分别介绍了基于内容的推荐方法和混合推荐策略；在此基础上，分析了推荐系统面临的三类关键问题——冷启动、兴趣漂移与流行度偏差，并给出了相应的解决思路；接着介绍了推荐系统常用的评价指标及其计算方法；最后概述了本系统实现所采用的技术栈，包括后端框架 Spring Boot 3 与 MyBatis、前端框架 Vue3 与 UniApp，以及 MySQL 与 Redis 数据存储方案。本章为后续第三章推荐算法的详细设计和第四章系统实现提供了必要的理论和技术基础。

第 3 章 算法原理与代码实现

3.1 算法总体设计

航空餐食推荐场景面临三重约束：航班餐食受航线绑定与舱位等级递进规则限制（头等舱可见全部舱位餐食、商务舱可见商务与经济舱、经济舱仅见经济舱）；用户口味存在短期漂移，静态画像难以捕捉；行为信号稀疏且类型多样（曝光、点击、预选、评分等）。针对上述特征，本文提出了一种三子模型融合的自适应混合推荐算法，算法标识为 `fused-pmfup-prmidm-ammbc-v3`。

算法由三个子模型构成：PMFUP（Preference Matching from User Profile）基于用户显式偏好进行内容匹配；PRMIDM（Preference Recent Migration and Interest Drift Model）通过对比近 30 天与近 180 天行为信号检测口味漂移；AMMBC（Adaptive Multi-Mode Behavior Collaborative）采用基于用户的协同过滤，引入收缩因子、流行度去偏与置信度加权。三子模型的融合权重由自适应解析器根据用户实时行为信号强度动态计算——信号稀疏时 PMFUP 占主导（默认权重 0.60），随信号积累协同过滤与漂移模型权重逐步提升。此外，针对多餐食航班引入前序餐食惩罚机制，避免相邻餐次重复推荐。

算法以一组静态常量集中管理核心参数，包括允许的餐食类型集合 `{1, 2, 3, 4}`（依次对应儿童餐、标准餐、清真餐、素食）、允许的口味标签集合 `{"清淡", "咸香", "微辣", "甜口", "低脂", "高蛋白"}`、舱位等级常量（`CABIN_FIRST=1`, `CABIN_BUSINESS=2`, `CABIN_ECONOMY=3`）、协同过滤收缩系数 `CF_SHRINK_LAMBDA=3.0`、近期窗口 `RECENT_WINDOW_DAYS=30`、历史窗口 `HISTORY_WINDOW_DAYS=180` 以及曝光信号基础权重 `EXPOSURE_SCORE_WEIGHT=0.15`。舱位等级递进规则 `resolveCabinCascade` 的实现逻辑为：若用户舱位为头等舱（1），返回可见类型 `[1, 2, 3]`；若为商务舱（2），返回 `[2, 3]`；若为经济舱（3）或空值，返回 `[3]`。该规则确保高舱位旅客可以浏览低舱位的餐食选项，反之则不可见。候选菜品通过 SQL 查询从 `flight_route_dish` 航线绑定表中筛选，限定条件为航线匹配、舱位等级可见（`cabin_type IN (...)` 递进规则）、库存大于零，单次最多返回 20 条。算法架构如图 3.1 所示。
==========todo

3.2 行为数据预处理

3.2.1 数据源与行为信号

算法数据源为 `recommendation_log` 表，每次推荐请求时加载近 180 天内最多 5000 条日志。每条日志包含：`user_id`（用户 ID）、`recommended_dishes`（推荐菜品 ID 列表，JSON 数组）、`user_feedback`（结构化行为反馈字符串）、`user_rating`（航后评分，1-5）和 `create_time`（创建时间）。

`user_feedback` 采用前缀格式记录用户行为：`CLICK:dishId=X:mealOrder=Y` 表示点击浏览，`MANUAL_SELECTED:dishId=X:mealOrder=Y` 表示手动预选，`AUTO_SELECTED_OVERDUE:dishId=X` 表示超时自动分配，`MANUAL_SELECTED_UPDATE:dishId=X:mealOrder=Y` 表示改选。其中 `dishId` 和 `mealOrder` 通过正则匹配从字符串中提取，`dishId` 优先以 `dishId=` 模式匹配，`mealOrder` 以 `mealOrder=` 模式匹配。`recommended_dishes` 字段存储为 JSON 数组（如 `[1, 3, 5]`），算法通过解析 JSON 并结合菜品名称到 ID 的映射表来还原完整的推荐菜品集合。

各类行为按重要性分配差异化的得分权重，并引入指数时间衰减函数加权：
：

$$decay(days) = e^{-0.018 \times days}$$

该函数半衰期约 38.5 天。表 3.1 给出了各行为类型的得分权重配置。

表 3.1：行为信号权重与口味漂移权重表

行为类型	feedback 前缀	得分权重	口味漂移权重
推荐曝光	recommended_dishes 列表	0.15	—
点击浏览	CLICK	2.0	1.4
手动预选	MANUAL_SELECTED	5.0	2.4
自动分配	AUTO_SELECTED_OVERDUE	3.0	1.8
航后高分评分	user_rating ≥ 4	2.0	1.4
航后低分评分	user_rating < 4	1.0	0.8

3.2.2 交互上下文构建

算法通过内部类 `InteractionContext` 在每次请求时批量构建共享数据源，包含六个核心字段：

```
private static class InteractionContext {
    final Map<Integer, Map<Integer, Double>> userDishScore;     // 用户-菜品累积得分
    final Map<Integer, Integer> dishExposureCount;              // 菜品曝光计数
    final Map<Integer, Integer> userBehaviorSignal;             // 用户行为信号数
    final Map<Integer, Map<String, Double>> userRecentFlavorScore; // 近 30 天口味得分
```

```
    final Map<Integer, Map<String, Double>> userLongFlavorScore;   // 近 180 天口味得分
    final Map<Integer, Set<String>> dishFlavorTags;            // 菜品口味标签
}
```

其中 `userRecentFlavorScore` 与 `userLongFlavorScore` 以 30 天为界划分，为 PRMIDM 子模型的漂移检测提供对比基础。`userBehaviorSignal` 统计的是"有行为信号的日志行数"——一条日志无论包含几种行为均计为 1。口味标签共有六种：清淡、咸香、微辣、甜口、低脂、高蛋白。

构建过程的核心逻辑以真实源码展示如下

```
for (Map<String, Object> row : logs) {
    double decay = timeDecay(parseDateTime(row.get("createTime")));
    // 1) 曝光信号：推荐列表中的每个菜品 +0.15
    for (Integer did : extractDishIds(row.get("recommendedDishes"))) {
        addScore(ctx.userDishScore, uid, did, 0.15 * decay);
        increaseDishExposure(ctx, did);
    }
    // 2) 行为信号：根据 feedback 前缀分配权重
    Integer selectedDishId = extractDishIdFromText(feedback);
    if (feedback.startsWith("MANUAL_SELECTED")) {
        addScore(ctx.userDishScore, uid, selectedDishId, 5.0 * decay);
        addFlavorDriftSignal(ctx, uid, selectedDishId, createTime, decay, 2.4);
    } else if (feedback.startsWith("AUTO_SELECTED_OVERDUE")) {
        addScore(ctx.userDishScore, uid, selectedDishId, 3.0 * decay);
        addFlavorDriftSignal(ctx, uid, selectedDishId, createTime, decay, 1.8);
    } else if (feedback.startsWith("CLICK")) {
        addScore(ctx.userDishScore, uid, selectedDishId, 2.0 * decay);
        addFlavorDriftSignal(ctx, uid, selectedDishId, createTime, decay, 1.4);
    }
    // 3) 评分信号：≥4 分权重 2.0，<4 分权重 1.0
    if (rating != null && rating > 0 && selectedDishId != null) {
        addScore(ctx.userDishScore, uid, selectedDishId, (rating >= 4 ? 2.0 : 1.0) * decay);
        addFlavorDriftSignal(ctx, uid, selectedDishId, createTime, decay, rating >= 4 ? 1.4 : 0.8);
    }
}
```

其中 `addFlavorDriftSignal` 根据 `createTime` 判定归属窗口：距当前 30 天内的归入 `userRecentFlavorScore`，30 至 180 天内的（或 `createTime` 为空时）归入 `userLongFlavorScore`。其实现逻辑为：根据日志创建时间与当前时间的天数差选择目标 Map，然后遍历菜品的每个口味标签，以 `weight * decay` 为增量累加到目标用户在目标口味上的得分中。`addScore` 采用累加模式（`map.merge(key, delta, Double::sum)`），而非覆盖，使得多次同类行为能够叠加体现持续偏好。辅助函数 `overlapCount` 通过遍历第一个 Map 的键集并检查第二个 Map 是否包含该键来统计共同键数量，在 AMMBC 子模型的收缩修正中用于计算两个用户行为向量的重叠度。

3.3 子模型一：PMFUP——基于用户画像的偏好匹配

PMFUP 子模型利用用户显式设定的偏好（餐食类型、口味标签）与菜品属性进行内容匹配，在行为信号稀疏时提供主要的推荐依据。菜品包含两种内容特征：`mealType`（1=儿童餐、2=标准餐、3=清真餐、4=素食）和 `flavorTags`（清淡、咸香、微辣、甜口、低脂、高蛋白中的若干标签）。

评分计算分三步：基础分统一为 0.2；若用户偏好集中的餐食类型与菜品匹配，追加奖励——单偏好时追加 0.45（明确偏好），多偏好时追加 0.38（宽泛偏好）；若口味标签匹配，设匹配数为 n，口味奖励为 $\min(0.35, 0.15 + n \times 0.10)$。最终得分钳位至 $[0,1]$：

$$S_{pmfup} = clamp(0.2 + typeBonus + flavorBonus)$$

真实源代码如下：

```
private double calculatePmfupScore(RecommendationDishVO item,
        Set<Integer> prefMealTypes, Set<String> prefFlavors, ...) {
    if (item == null || item.getDishId() == null) return 0.0;
    double prefScore = 0.2;
    if (!prefMealTypes.isEmpty() && item.getMealType() != null
            && prefMealTypes.contains(item.getMealType())) {
        prefScore += prefMealTypes.size() == 1 ? 0.45 : 0.38;
    }
    if (!prefFlavors.isEmpty() && item.getFlavorTags() != null) {
        int matched = 0;
        for (String flavor : prefFlavors) {
            if (item.getFlavorTags().contains(flavor)) matched++;
        }
        if (matched > 0) {
            prefScore += Math.min(0.35, 0.15 + matched * 0.10);
        }
    }
    return clamp01(prefScore);
}
```

PMFUP 得分不依赖航线、时间与行为关联数据，保证了冷启动场景下的可用性。其本质是用户显式声明的偏好与菜品属性标签之间的集合匹配，匹配粒度越细（口味标签重合越多），得分越高，但受上限 0.35 约束以避免口味匹配过度主导总分。餐食类型匹配的奖励差异化（0.45 vs 0.38）则反映了偏好明确程度的区分：单一偏好意味着用户决策聚焦，匹配应获更高权重。

以具体数值为例：设用户仅偏好标准餐（mealType=2）且偏好口味为 `["清淡", "低脂", "高蛋白"]`（共 3 种）。对一道标准餐、口味标签为 `["清淡", "低脂", "高蛋白"]` 的"低脂鸡胸藜麦饭"，计算过程为：基础分 0.2，餐食类型匹配（单偏好）追加 0.45，口味匹配数 $n = 3$，口味奖励 $\min(0.35, 0.15 + 3 \times 0.10) = 0.35$（触及上限），最终得分 $0.2 + 0.45 + 0.35 = 1.0$。对一道标准餐、口味仅为 `["咸香","热食"]` 的"经典红烧牛腩饭"：基础分 0.2，类型匹配+0.45，口味匹配数 $n = 0$，口味奖励 0，最终得分 0.65。两者差距体现了口味标签匹配对推荐的区分能力。

3.4 子模型二：PRMIDM——偏好漂移与兴趣迁移模型

RMIDM 子模型通过对比近期（30 天）与历史（180 天）的口味偏好信号，检测用户口味的短期漂移方向。对于候选菜品 d，计算步骤如下。

（1）从交互上下文的 `userRecentFlavorScore` 和 `userLongFlavorScore` 中，汇总用户对菜品所含各口味的近期得分 R_f 与历史得分 H_f：

$$recentScore = \sum_f R_f, \quad historyScore = \sum_f H_f$$

（2）采用 sigmoid 型归一化函数将两类得分映射至 $[0,1]$。归一化函数的选择需满足三个条件：在 $x = 0$ 附近近似线性以保持小信号的区分度，在 x 较大时趋于饱和以避免少数极端值过度支配，值域为 $[0,1)$。本文采用的归一化公式为：

$$norm(x) = \frac{x}{x + 2.0}$$

其实现代码为 `private double normalizeSignal(double score) { return clamp01(score / (score + 2.0)); }`，其中常数 2.0 控制饱和速度：$x=2$ 时归一化值为 0.5，$x=8$ 时约为 0.8。将两类得分分别归一化：

$$recentPref = norm(recentScore), \quad historyPref = norm(historyScore)$$

（3）漂移增量定义为近期偏好减去历史偏好的正向差值：$driftBoost = clamp(recentPref - historyPref)$，仅当近期偏好显著高于历史时才为正。同时获取用户对该菜品的直接兴趣得分 $interest$，计算短期菜品信号 $dishSignal = clamp(interest/8.0)$。

（4）加权合成：

$$S_{prmidm} = clamp(0.60 \times recentPref + 0.30 \times driftBoost + 0.10 \times dishSignal)$$

真实源代码如下：

```
private double calculatePrmidmScore(Integer userId, Integer dishId, InteractionContext ctx) {
    if (dishId == null || userId == null) return 0.0;
    Set<String> targetFlavors = ctx.dishFlavorTags.getOrDefault(dishId, Collections.emptySet());
    Map<String, Double> recentFlavor = ctx.userRecentFlavorScore.getOrDefault(userId,
Collections.emptyMap());
    Map<String, Double> longFlavor = ctx.userLongFlavorScore.getOrDefault(userId, Collections.emptyMap());
    double recentScore = 0.0, historyScore = 0.0;
    for (String flavor : targetFlavors) {
        recentScore += recentFlavor.getOrDefault(flavor, 0.0);
        historyScore += longFlavor.getOrDefault(flavor, 0.0);
    }
    double interest = ctx.userDishScore.getOrDefault(userId, Collections.emptyMap())
            .getOrDefault(dishId, 0.0);
    double recentPref = normalizeSignal(recentScore);
    double historyPref = normalizeSignal(historyScore);
    double driftBoost = clamp01(recentPref - historyPref);
    double shortTermSignal = clamp01(interest / 8.0);
    return clamp01(0.60 * recentPref + 0.30 * driftBoost + 0.10 * shortTermSignal);
}
```

3.5 子模型三：AMMBC——自适应协同过滤

AMMBC 子模型采用基于用户的协同过滤，在标准余弦相似度的基础上引入三种修正机制：收缩因子修正小重叠度下的假性高相似，流行度去偏缓解热门菜品的马太效应，置信度加权在邻居证据不足时降低预测可信度。余弦相似度采用单遍点积方式实现，适用于稀疏的用户-菜品得分向量——遍历第一个向量的键集同时累加点积和第一个范数，再单独累加第二个向量的范数，最后以点积除以两个范数平方根的乘积。算法对每个有效邻居（行为向量中包含目标菜品的其他用户）执行以下步骤：计算余弦相似度；统计两用户行为向量的共同键数量作为重叠度；应用收缩因子修正相似度；将修正相似度乘以邻居对该菜品的得分，累加到加权预测值中。

收缩因子修正：设两个用户共同交互的菜品数量为 $overlap$，收缩系数 $\lambda = 3.0$，修正后的相似度为：

$$adjustedSim = sim \times \frac{overlap}{overlap + 3.0}$$

该修正的效果可通过数值说明：$overlap = 1$ 时修正系数为 0.25，相似度被大幅削弱，避免因偶然的单一共同菜品产生假性高相似；$overlap = 3$ 时修正系数为 0.5，相似度折半保留；$overlap = 10$ 时修正系数约为 0.77，逐步逼近原始相似度。该机制使得两个用户只有在拥有充分共同交互证据时才被认定为高度相似。

流行度去偏：设菜品累计曝光次数为 exp，惩罚系数为：

$$popDebias = \frac{1.0}{1.0 + \frac{\ln(1 + exp)}{6.0}}$$

置信度加权：设邻居相似度总量为 $simTotal$，邻居数为 $neighborCount$，置信度定义为：

$$confidence = clamp\left(\frac{simTotal}{\max(1.0, neighborCount \times 0.45)}\right)$$

该公式的含义为：当每个邻居的平均修正相似度达到 0.45 时置信度为 1.0；若邻居的平均相似度偏低（如多为 0.2 左右的弱关联用户），置信度按比例降低。置信度综合反映了"邻居群体在统计意义上是否足够可靠"。例如，3 个邻居每人平均相似度仅 0.2，则 $simTotal = 0.6$，$confidence = 0.6/\max(1.0, 3 \times 0.45) = 0.6/1.35 \approx 0.44$，协同项的权重系数 $0.70 + 0.30 \times 0.44 = 0.83$，协同得分被适度降低。

AMMBC 最终得分由协同预测分与三种修正因子合成：

$$S_{ammbc} = clamp\big(collaborative \times popDebias \times (0.70 + 0.30 \times confidence)\big)$$

其中 $collaborative = clamp\left(\frac{simWeighted/simTotal}{8.0}\right)$，系数 8.0 为归一化缩放因子——当加权平均预测分达到 8.0 时 $collaborative$ 饱和为 1.0。$simWeighted = \sum adjustedSim \times r(v, d)$ 为邻居对目标菜品的修正相似度加权累积。核心实现逻辑为：遍历 `userDishScore` 中所有其他用户，跳过自身及未包含目标菜品的用户后，依次计算余弦相似度、统计重叠度、计算收缩修正相似度，将有效邻居的修正相似度加权累加；若最终相似度总量为零则返回 0.0，否则依次计算协同预测分、流行度去偏系数和置信度，合成最终得分。

3.6 自适应权重融合与多餐约束

3.6.1 融合权重自适应调整

设用户行为信号数为 s，行为覆盖菜品数为 c，口味信号总量为 d，定义三个中间度量：

$$signalConf = clamp\left(\frac{\ln(1+s)}{\ln 15}\right),$$
$$behavCover = clamp\left(\frac{c}{10.0}\right),$$
$$driftEvid = clamp\left(\frac{d}{12.0}\right)$$

权重原始值的计算公式设计为：PMFUP 基础最高（0.46），随信号增长递减；PRMIDM 受漂移证据驱动；AMMBC 随信号与覆盖度增长。真实源码如下：

```
private double[] resolveFusionWeights(Integer userId, InteractionContext ctx) {
    if (userId == null || ctx == null) return new double[]{0.60, 0.25, 0.15};
    int signalCount = ctx.userBehaviorSignal.getOrDefault(userId, 0);
    double signalConf = clamp01(Math.log1p(signalCount) / Math.log(15.0));
    int coverageSize = ctx.userDishScore.getOrDefault(userId, Collections.emptyMap()).size();
    double behavCover = clamp01(coverageSize / 10.0);
    double recentSig = sumSignal(ctx.userRecentFlavorScore.getOrDefault(userId, Collections.emptyMap()));
    double historySig = sumSignal(ctx.userLongFlavorScore.getOrDefault(userId, Collections.emptyMap()));
    double driftEvid = clamp01((recentSig + historySig) / 12.0);
    double w1 = 0.46 + 0.22 * (1 - signalConf) + 0.08 * (1 - driftEvid);
    double w2 = 0.22 + 0.30 * driftEvid + 0.12 * signalConf;
    double w3 = 0.16 + 0.26 * signalConf + 0.18 * behavCover;
    double total = w1 + w2 + w3;
    if (total <= 0) return new double[]{0.60, 0.25, 0.15};
    return new double[]{w1 / total, w2 / total, w3 / total};
}
```

融合得分为 $S_{fused} = w_1 S_{pmfup} + w_2 S_{prmidm} + w_3 S_{ammbc}$。默认权重 [0.60,0.25,0.15] 体现了"行为信号缺失时优先依赖画像匹配"的原则。以新用户冷启动场景为例：$s = 0$、$c = 0$、$d \approx 0$，代入得 $signalConf = 0$、$behavCover = 0$、$driftEvid \approx 0$，$w_1^{raw} = 0.46 + 0.22 + 0.08 = 0.76$，$w_2^{raw} = 0.22$，$w_3^{raw} = 0.16$，归一化后约为 [0.67,0.19,0.14]，接近默认值，PMFUP 主导。当用户积累 8 条信号、覆盖 7 种菜品后：$signalConf \approx 0.81$，$behavCover = 0.7$，$w_1^{raw} = 0.46 + 0.22 \times 0.19 + 0.08 \times (1-d)$、$w_2^{raw} = 0.22 + 0.30d + 0.12 \times 0.81$、$w_3^{raw} = 0.16 + 0.26 \times 0.81 + 0.18 \times 0.7 \approx 0.50$，三权重趋于均衡。当用户行为信号极为丰富（$s \geq 14, c \geq 10$）时，$signalConf \approx 1.0$，$behavCover = 1.0$，AMMBC 的原始权重可增长至 $0.16 + 0.26 + 0.18 = 0.60$，归一化后三个子模型权重大约各占三分之一。这一自适应的设计使得算法无需人工干预即可在冷启动到充分个性化的全生命周期中平滑运行。

表 3.2：不同行为信号强度下的融合权重示例

场景	signalCount	覆盖菜品数	PMFUP 权重	PRMIDM 权重	AMMBC 权重
新用户冷启动	0	0	~0.60	~0.25	~0.15
少量交互	3	3	~0.4	~0.28	~0.24
中等交互	8	7	~0.39	~0.31	~0.30
丰富交互	15+	10	~0.33	~0.33	~0.34

3.6.2 多餐前序惩罚与解释性输出

对于第n餐（$n \geq 2$），查询用户在前序餐次 $n-1$ 的选择记录。实现方式为：通过 `recommendationMapper.latestManualSelection(userId, flightId, mealOrder-1)` 获取前序餐次的预选记录，提取 `prevDishId` 及其 `mealType`，然后在逐菜品评分循环中判断——若当前菜品 ID 等于 `prevDishId` 则乘以 0.70，否则若菜品 `mealType` 与 `prevMealType` 相同则乘以 0.85。该机制不阻断任何菜品的推荐——惩罚后仍可出现在推荐列表中（仅为降权），在"避免重复"与"保留选择权"之间取得平衡。

推荐结果附带可解释标签，各子模型的触发阈值及对应标签如表 3.3 所示。

3.3：推荐解释标签触发条件

解释标签	触发条件	来源子模型
画像偏好匹配	$S_{pmfup} \geq 0.65$	PMFUP
近期口味漂移	$S_{prmidm} \geq 0.55$	PRMIDM
协同过滤匹配	$S_{ammbc} \geq 0.40$	AMMBC
行为样本不足	$signalCount < 3$	通用
与前序餐食相同	触发前序惩罚	通用
基础稳健推荐	以上均不满足	通用

3.7 算法总体流程

综合以上各节，算法完整流程的伪代码如算法 1 所示。

****算法 1：Fused-PMFUP-PRMIDM-AMMBC 混合推荐算法****

```
输入: userId, flightId, mealOrder
输出: 个性化推荐列表 (含得分与解释标签)

flight ← getFlightInfo(flightId)
cabinTypes ← resolveCabinCascade(user.cabinType)
    // 头等舱→[1,2,3], 商务舱→[2,3], 经济舱→[3]
candidates ← listCandidateDishes(flight.departure, flight.destination,
    cabinTypes, limit=20)  // SQL 查询，含航线绑定与库存过滤
if candidates 为空: return []

logs ← listRecentLogs(180 天, 5000 条)
context ← buildInteractionContext(logs)     // 3.2 节, 构建共享数据源
[w1,w2,w3] ← resolveFusionWeights(userId, context)  // 3.6.1 节, 自适应权重
prevDish ← findPreviousSelection(userId, flightId, mealOrder-1)

for each dish in candidates:
    S1 ← calculatePmfupScore(dish, userPrefs)    // 3.3 节
    S2 ← calculatePrmidmScore(userId, dish.id, context)  // 3.4 节
    S3 ← calculateAmmbcScore(userId, dish.id, context)   // 3.5 节
    fused ← w1*S1 + w2*S2 + w3*S3
    fused ← applySequentialPenalty(fused, dish, prevDish)  // 3.6.2 节
    dish.score ← clamp01(fused)
    dish.reason ← generateExplain(S1, S2, S3, signalCount) // 3.6.2 节

sort candidates by score descending
insertRecommendationLog(userId, flightId, candidates)
```

```
22: return candidates
```

```

时间复杂度分析：候选查询阶段通过单次 SQL 完成，复杂度 $O(|D|)$；上下文构建阶段遍历最多 5000 条日志及每条日志的推荐列表，复杂度 $O(|L| \times |R|)$；评分阶段中，PMFUP 为 $O(|C|)$，PRMIDM 为 $O(|C| \times |F|)$（$|F| \leq 6$），AMMBC 需遍历所有用户计算余弦相似度，为主要计算开销 $O(|U| \times |D_u|)$。在 90 用户、10 菜品的测试数据集上，单次推荐延时为毫秒级，满足实时要求。算法中复用的四个辅助函数均为无外部库依赖的简单实现：`timeDecay` 计算指数衰减 $e^{-0.018 \times days}$（时间为空时返回 0.8）；`clamp01` 将值钳位至 [0,1]；`sumSignal` 对口味得分 Map 中的所有正值求和（用于权重解析时计算漂移证据总量）；`normalizeSignal` 执行 sigmoid 型归一化 $x/(x+2.0)$（用于 PRMIDM 的偏好得分标准化）。



# 第 4 章 系统设计与实现

## 4.1 开发环境与工具

本系统的开发、构建与运行环境如表 4.1 所示。

表 4.1：开发环境配置

| 类别 | 工具/框架 | 版本 |
|---|---|---|
| 后端框架 | Spring Boot | 3. |
| JDK | Java | 17 |
| 持久层框架 | MyBatis Spring Boot Starter | 3.0.3 |
| 关系数据库 | MySQL | 8.x |
| 缓存数据库 | Redis | — |
| 构建工具 | Maven | 3.9+ |
| 管理端框架 | Vue + TypeScript | 3.4.21 / 5.4.0 |
| UI 组件库 | Element Plus | 2.7.1 |
| 数据可视化 | ECharts | 5.5.0 |
| 状态管理 | Pinia | 2.1.7 |
| 构建工具 | Vite | 5.2.8 |
| 小程序框架 | UniApp (Vue 3) | 3.0.0 |
| 微信开发者工具 | 微信开发者工具 | — |
| 集成开发环境 | IntelliJ IDEA / VS Code | — |
| 操作系统 | Windows 11 | — \| |

### 4.2 系统架构设计

#### 4.2.1 总体架构

本系统采用前后端分离的 B/S 架构[14][15]，整体分为三层：前端展示层、后端服务层和数据存储层。前端展示层包含面向运营管理人员的管理端（Vue3 单页应用）和面向航空旅客的小程序端（UniApp 跨端应用），两者通过 HTTP 协议向后端服务层发送 RESTful API 请求。后端服务层基
```

于 Spring Boot 3 构建，以控制层（Controller）接收请求，通过业务逻辑层完成数据处理与推荐算法运算，经数据访问层（MyBatis Mapper）与数据库交互。数据存储层采用 MySQL 作为主数据库存储业务数据，Redis 作为缓存数据库存储热点数据。

系统通过 Nginx[14]实现前端静态资源部署与 API 反向代理。管理端打包为静态文件部署于 Nginx，小程序端编译为微信小程序包上传至微信平台，后端 Spring Boot 应用以内嵌 Tomcat 方式独立运行于服务器 8081 端口。系统总体架构如图 4.1 所示。

> **[此处插入图 4.1：系统总体架构图（三层架构：前端展示层→后端服务层→数据存储层）]**

4.2.2 后端分层架构

后端 Spring Boot 应用内部遵循"控制层—业务层—数据访问层"三层分离原则，具体分包结构如下：

- `controller/admin/`：管理端控制器，包含 EmployeeController（员工管理）、DishController（菜品管理）、FlightController（航班管理）、AnnouncementController（公告与推荐看板）、UserMealCenterController（用户选餐统计）等；
- `controller/user/`：用户端控制器，包含 UserController（用户信息与偏好）、RecommendationController（推荐引擎与评分）；
- `mapper/`：MyBatis 数据访问接口，每个 Mapper 对应一组 SQL 映射，共 11 个 Mapper 接口；
- `task/`：定时任务组件，包含 MealSelectionTask（预选超时自动分配与截止提醒）；
- `entity/`：实体类，与数据库表一一映射；
- `vo/`与`dto/`：视图对象与数据传输对象，用于 API 响应和请求参数封装。

> **[此处插入图 4.2：后端分包结构图]**

4.3 数据库设计

本系统数据库名为`aviation_food_recommend`，核心业务涉及 18 张数据表，按功能划分为五组：用户与员工（`user`、`user_preference`、`employee`）、航班与航线（`flight_info`、`flight_route_dish`）、菜品与分类（`category`、`dish`、`dish_flavor`等）、推荐与预选（`recommendation_log`、`meal_selection`）以及评分与公告（`flight_service_rating`、`flight_announcement`）。核心表关系 E-R 图如图 4.3 所示，航线-菜品绑定表`flight_route_dish`的结构如表 4.2 所示。

表 4.2：flight_route_dish 航线菜品绑定表结构

字段名	类型	约束	说明
id	BIGINT	PK	主键
departure	VARCHAR(50)	NOT NULL	起飞机场
destination	VARCHAR(50)	NOT NULL	目的地机场
dish_id	BIGINT	NOT NULL	菜品 ID
cabin_type	TINYINT	DEFAULT 3	舱位类型（1 头等/2 商务/3 经济）

> **[此处插入图 4.3：数据库核心表 E-R 图]**

4.4 系统功能模块设计

系统按用户角色划分为管理端和小程序端两大功能域。管理端面向航空公司运营人员，包含航空总览、航班运营中心、航班管理中心、航班餐食中心、餐食资源中心、用户餐食中心、公告管理中心、评分管理中心和人员管理九个模块。小程序端面向航空旅客，形成"微信登录→航班绑定→偏好采集→餐食预选→航后评分"的五步用户闭环。各域内部的模块划分如图 4.4 所示。

> **[此处插入图 4.4：系统功能模块图（左右分栏：管理端 9 大模块 + 小程序端用户闭环 5 步）]**

4.5 管理端实现

4.5.1 登录与权限管理

管理端登录页采用全屏背景配合居中登录卡片的视觉设计。员工提交账号密码后，后端通过`POST /admin/employee/login`验证并签发 JWT 令牌，前端存储于 Pinia 状态及本地缓存，后续请求经由 Axios 拦截器自动携带令牌完成认证。登录后进入主界面，左侧为九大功能模块的垂直导航菜单，右侧为对应的内容展示区。管理端主界面如图 4.5 所示。

> **[此处插入图 4.5：管理端主界面（航空总览看板页）]**

4.5.2 航空总览看板

航空总览页面作为管理端首页，以数据看板形式呈现推荐系统的核心运营指标。页面顶部为四个统计卡片，分别展示推荐总次数、评分均值、热门菜品数和异常用户数。中部为 ECharts 渲染的近期推荐趋势折线图与热门菜品柱状图。底部为推荐异常用户表格，列出偏好未完成或未绑定航班的用户。后端通过`GET /admin/recommendation/dashboard`和`GET /admin/recommendation/top`接口返回统计数据。

4.5.3 航班管理与餐食绑定

航班管理中心以表格形式列出所有航班，支持按航班号搜索。新增航班时填写航班号、起降地、起降时间（日期时间选择器）、飞行时长（分钟）、餐食数量与餐食时段（JSON 数组，如`["午餐","加餐"]`）以及预选截止时间。编辑和删除操作逐行提供。

航班餐食中心实现航线-菜品绑定功能。选择航班后，页面展示该航班当前已绑定的菜品列表（含菜品名称、舱位类型、排序）。点击"绑定餐食"弹出对话框，选择菜品并指定舱位类型（头等舱/商务舱/经济舱），通过`POST /admin/flight/meal`提交绑定。舱位递进规则在 SQL 查询层生效：查询时以`cabin_type IN (...)`条件实现高舱位可见低舱位餐食。航班餐食绑定界面如图 4.6 所示。

> **[此处插入图 4.6：航班餐食中心页面（左侧航班列表 + 右侧已绑定菜品表 + 绑定操作按钮）]**

4.5.4 餐食资源管理

餐食资源中心以表格展示所有菜品，支持按名称和分类筛选。新增菜品时需填写名称、图片、描述、所属分类、餐食类型（mealType，四选一）、口味标签（flavorTags，六种多选）和库存数量。后端`POST /admin/dish`同步写入`dish`与`dish_flavor`表。

> **[此处插入图 4.7：餐食资源中心页面（菜品表格 + 新增/编辑菜品对话框含餐食类型和口味标签字段）]**

4.5.5 评分管理与数据监控

评分管理中心以表格展示所有评分任务，支持按评分状态（PENDING/SUBMITTED/DEFERRED/EXPIRED）和航班号筛选。每条记录显示用户姓名、航班号、评分分数、评分状态、首次可见时间、延期次数与过期时间。管理员可对特定任务执行重开（`POST /admin/recommendation/rating/{id}/reopen`）或手动过期（`POST /admin/recommendation/rating/{id}/expire`）操作。评分管理界面如图 4.8 所示。

> **[此处插入图 4.8：评分管理中心页面（筛选栏 + 评分任务表格 + 状态标签 + 操作按钮）]**

用户餐食中心展示旅客的餐食预选记录（含用户姓名、身份证号、航班号、预选状态、菜品名称、口味与数量），支持按航班号统计各菜品的预选需求量分布，辅助配餐决策。后端通过`GET /admin/user-meal/list`和`GET /admin/user-meal/statistics`提供数据。

4.6 小程序端实现

4.6.1 微信登录与身份核验

小程序启动时调用`wx.login`获取临时 code，前端将 code 发送至`POST /user/user/login`，后端通过微信 API 以 code 换取用户的 OpenID，查询或新建用户记录后签发 JWT 令牌（HS256 签名，有效期 10 天）。令牌由前端 Pinia 持久化存储，后续请求通过拦截器自动注入 Header。

身份核验页面要求旅客输入 18 位身份证号，前端以正则表达式`/^[1-9]\d{5}(18|19|20)\d{2}(0[1-9]|1[0-2])(0[1-9]|[12]\d|3[01])\d{3}[0-9Xx]$/`校验格式，通过后调用`PUT /user/user`更新身份证号。页面同时展示旅客的舱位类型（由管理员在后台设定，旅客不可自行修改）及舱位递进规则说明。微信登录与身份核验界面如图 4.9 所示。

> **[此处插入图 4.9：小程序登录页 + 身份核验页截图]**

4.6.2 航班绑定

身份核验完成后，前端通过`GET /user/flight/list`获取与该身份证号关联的候选航班列表。页面以卡片形式展示每个航班的航班号、起降地、起飞时间、餐食数量和预选截止时间。旅客点击"绑定"按钮调用`POST /user/flight/bind`将航班与用户关联（更新`user.current_flight_id`）。绑定后，页面顶部展示当前主航班信息及选餐状态，底部提供"进入餐食预选"按钮跳转至推荐页。

4.6.3 偏好采集

首次进入偏好采集页时，页面顶部展示欢迎引导语。偏好设置包含两类选项：餐食类型单选（儿童餐/标准餐/清真餐/素食）和口味标签多选（清淡、咸香、微辣、甜口、低脂、高蛋白，以标签网格展示，点击切换选中状态）。旅客可点击"保存偏好并进入推荐"提交数据（`PUT /user/preference`，将选定值序列化为 JSON 后发送），也可点击"先跳过，直接体验推荐"在未设偏好时进入推荐页——此时 PMFUP 子模型仅使用基础分 0.2，推荐结果依赖 PRMIDM 与 AMMBC 的行为反馈（若新用户无任何历史行为则大幅依赖默认权重下的内容兜底）。

4.6.4 餐食预选与推荐

餐食预选页是本系统的核心用户界面，由四个功能区域组成。顶部为航班选择区和预选截止时间提示；其下为选餐状态卡片，展示当前餐次、已完成的预选进度及截止状态；中间筛选栏提供口味与餐食类型过滤条件，多餐食航班额外显示餐次切换器；底部推荐列表以水平滑动卡片展示菜品，每张卡片含菜品图片、得分进度条、口味标签和推荐理由。卡片操作按钮根据预选状态动态切换——未预选时显示"选择并确认"，已预选时显示"改选并确认"并标记"当前已选"，截止后按钮禁用。点击进入确认页，确认后调用`POST /user/recommendation/select`提交预选、扣减库存并记录行为反馈。餐食预选界面如图 4.10 所示。

> **[此处插入图 4.10：餐食预选推荐页（航班选择+状态卡+筛选栏+菜品卡片）]**

4.6.5 航后评分

航班落地后，系统根据`meal_selection`表中的手动预选记录自动生成评分任务（写入`flight_service_rating`表，状态为 PENDING）。评分页以列表展示所有待评分任务，每条包含航班号、起降地、到达时间、已选菜品名称和评分状态。旅客点击某条任务进入评分界面，以五星评分组件对菜品进行 1-5 分评价，提交后调用`POST /user/recommendation/rate`，任务状态更新为 SUBMITTED。旅客也可选择"稍后评价"调用`POST /user/recommendation/rate/defer`将任务推迟 24 小时后再提醒。若 7 天内未提交评分，定时任务将任务状态自动变更为 EXPIRED。航后评分界面如图 4.11 所示。

> **[此处插入图 4.11：航后评分页面（评分任务列表 + 五星评分组件 + 暂缓按钮）]**

4.7 后端关键服务实现

4.7.1 JWT 认证与拦截器

系统通过 JwtInterceptor 拦截器实现统一身份认证。管理端拦截器拦截`/admin/**`路径（排除`/admin/employee/login`），从请求头`token`字段提取 JWT 并验证签名和有效期。用户端拦截器拦截`/user/**`路径（排除`/user/user/login`），从请求头`authentication`字段提取 JWT。两个拦截器使用不同

的签名密钥（管理端`hyk_handsome`，用户端`user_handsome`）和令牌有效期配置。JWT 工具类基于`io.jsonwebtoken:jjwt:0.9.1`实现，提供`createJWT()`和`parseJWT()`静态方法，签名算法为 HS256。

4.7.2 推荐引擎接口

推荐引擎对外暴露的 REST API 端点及功能如表 4.3 所示。

方法	URL 路径	功能
GET	`/user/recommendation/list`	获取个性化推荐列表，参数：mealType, flavor, mealOrder, size
POST	`/user/recommendation/click`	上报菜品点击行为
POST	`/user/recommendation/select`	确认菜品预选，扣减库存
GET	`/user/recommendation/history`	查询推荐历史记录
GET	`/user/recommendation/history/{logId}/breakdown`	查询某次推荐的三子模型得分明细
GET	`/user/recommendation/top`	获取热门菜品排名
GET	`/user/recommendation/pending-rating`	获取待评分任务列表
POST	`/user/recommendation/rate`	提交航后评分
POST	`/user/recommendation/rate/defer`	暂缓评分（推迟 24 小时）

推荐引擎的完整算法设计已在第三章详述，本章不再重复。后端核心业务逻辑集中在`RecommendationController.java`（约 1326 行），该控制器集成了候选菜品筛选、交互上下文构建、三子模型评分、自适应用权重融合、前序惩罚、解释生成与日志记录的全链路推荐流程。

4.7.3 预选定时任务

`MealSelectionTask`组件包含两个基于 Spring `@Scheduled`注解的定时任务。

自动分配任务（`autoSelectWhenOverdue`，Cron 表达式`0 */5 * * * ?`，每 5 分钟执行）：扫描所有状态为启用且预选截止时间已过但出发时间未到的航班，对每个航班中尚未手动预选的旅客执行自动菜品分配。分配逻辑为：读取旅客的餐食偏好，查询该航线该舱位下可用的候选菜品列表，按偏好匹配优先级选择菜品，尝试原子扣减库存（SQL `UPDATE dish SET stock = stock - 1 WHERE stock > 0`），若扣减成功则写入`meal_selection`表（状态=5 表示自动分配，编号前缀"AUTO"），并在推荐日志中写入`AUTO_SELECTED_OVERDUE`反馈。

截止提醒任务（`pushSelectionReminder`，Cron 表达式`0 */10 * * * ?`，每 10 分钟执行）：对所有启用航班计算距计划起飞时间的剩余小时数。在起飞前 12 小时和起飞前 3 小时两个时间节点，自动向航班写入提醒公告（通过`flight_announcement`表），公告内容包含航班号和预选截止时间提示。旅客可通过小程序公告页查看。

4.7.4 评分生命周期管理

评分任务包含四种状态：PENDING（待评分）、SUBMITTED（已提交）、DEFERRED（已暂缓）和 EXPIRED（已过期）。航班结束后，系统根据`meal_selection`中的手动预选记录批量生成 PENDING 任务（过期时间 7 天）。旅客提交评分后转为 SUBMITTED；暂缓则转为 DEFERRED 并在 24 小时后重新提醒；7 天内未提交自动过期。管理员可通过后台重开过期任务。状态流转如图 4.12 所示。

评分任务的状态流转如图 4.12 所示。

> **[此处插入图 4.12：评分任务状态流转图（PENDING→SUBMITTED/DEFERRED→EXPIRED，含重开回路）]**

4.8 本章小结

本章从开发环境、系统架构、数据库设计及功能模块实现四个方面阐述了航空旅客智能美食推荐系统的设计与实现。系统采用前后端分离架构，管理端基于 Vue3 + Element Plus 实现九大功能模块，小程

序端基于 UniApp 实现五步用户闭环，后端基于 Spring Boot 3 提供 65 个 REST API 端点并辅以定时任务完成预选超时自动分配与评分生命周期管理。

第 5 章 系统测试与结果分析

5.1 测试环境

本系统测试环境为 Windows 11 操作系统，硬件配置为 Intel Core i7 处理器、16 GB 内存。后端基于 Spring Boot 3.2.5 + JDK 21 + Maven 3.9 构建，数据库使用 MySQL 8.0 与 Redis 7.x。测试框架采用 JUnit 5 结合 Mockito 进行单元测试，通过 `@SpringBootTest`注解启动完整 Spring 上下文进行集成测试。管理端基于 Vue 3.4.21 + Vite 5.2.8 运行，小程序端基于 UniApp 3.0.0 编译为微信小程序并通过微信开发者工具调试。全部 29 个测试用例均通过，无失败或跳过。

5.2 测试数据构建

由于航空餐食推荐缺乏公开的真实用户行为数据集，本系统通过 SQL 脚本（`fused-recommend-demo-data.sql`与`fused-recommend-defense-boost.sql`）构造了模拟数据集。数据集包含 90 名用户、10 道菜品、4 个航班、12 条航线-菜品绑定记录以及 309 条推荐日志。用户按舱位分布为头等舱约 10%（9 人）、商务舱约 10%（9 人）、经济舱约 80%（72 人），偏好完成率约 92%（约 8%未完成以模拟冷启动用户）。10 道菜品涵盖 5 种分类，覆盖全部 4 种餐食类型和 6 种口味标签，库存量在 50 至 90 份之间。4 个航班分别处于不同运营状态：FUS6101（上海→成都，含午餐与加餐两个餐次，预选开放中）、FUS6202（北京→深圳，预选已超时）、FUS6303（广州→乌鲁木齐，已抵达可评分）、FUS6404（杭州→西安，12 天前的历史航班，用于行为信号时间衰减验证）。

5.3 推荐算法效果分析

5.3.1 融合权重自适应验证

融合权重自适应机制通过`RecommendationControllerTest`中的三个单元测试进行验证。`resolveFusionWeights_shouldPreferPmfupWhenBehaviorSignalsSparse`验证了行为信号稀疏时 PMFUP 权重占主导；
`resolveFusionWeights_shouldChangeSmoothlyWithSignalGrowth`验证了权重随信号增长平滑变化；
`resolveFusionWeights_shouldKeepDefaultWhenBehaviorSignalsRich`验证了信号丰富时三权重趋于均衡。三个测试均通过，确认了权重自适应逻辑从"冷启动依赖 PMFUP"到"丰富信号三模型均衡"的设计目标。在 5.3.2 节的评分分解实例中，userId=10 用户的权重为 PMFUP=0.362、PRMIDM=0.371、AMMBC=0.267，处于中等交互水平下的均衡状态，与自适应公式的预期一致。

5.3.2 三子模型评分分解

为直观展示三个子模型的分工与互补效果，本系统提供了推荐历史评分分解接口`GET /user/recommendation/history/{logId}/breakdown`，返回三子模型的独立得分与融合得分，用于定性分析和人工验证。以下是 userId=10 用户在 FUS6101 航班上的一次实际推荐输出（权重：PMFUP=0.362，PRMIDM=0.371，AMMBC=0.267）：

```
=== Recommendation Breakdown For Log 539 ===
userId=10, flightId=1, dishes=[2, 4, 3, 1]
weights: pmfup=0.361870, prmidm=0.370832, ammbc=0.267298

dishId   dishName            pmfup      prmidm     ammbc      fused      per
cent
2          经典红烧牛腩饭       0.650000 0.896684 0.328226 0.655469 65.55%
4          川香麻辣牛肉面       0.900000 0.772300 0.167355 0.656810 65.68%
3          香烤鳕鱼时蔬饭       0.900000 0.059430 0.160084 0.390512 39.05%
1          低脂鸡胸藜麦饭       0.900000 0.062703 0.119260 0.380813 38.08%
```

表 5.1：三子模型评分分解

菜品	PMFUP	PRMIDM	AMMBC	融合得分	推荐理由
川香麻辣牛肉面	0.900	0.772	0.167	**0.657**	画像偏好匹配 + 近期口味漂移
经典红烧牛腩饭	0.650	0.897	0.328	**0.655**	近期口味漂移 + 协同过滤匹配
香烤鳕鱼时蔬饭	0.900	0.059	0.160	**0.391**	画像偏好匹配
低脂鸡胸藜麦饭	0.900	0.063	0.119	**0.381**	画像偏好匹配

由表 5.1 可以分析出三个子模型的协同工作机制：

（1）**PMFUP 的锚定作用**：四道菜品均为标准餐类型，与用户偏好匹配，PMFUP 得分普遍在 0.65 以上，为推荐提供了稳定的内容基线。

（2）**PRMIDM 的区分作用**：麻辣牛肉面（0.772）和红烧牛腩饭（0.897）的 PRMIDM 得分显著高于鳕鱼饭（0.059）和藜麦饭（0.063）。该用户的近期行为信号对"咸香""微辣"口味存在正向漂移，而对"清淡""低脂"口味近期信号较弱，PRMIDM 准确捕捉了这一漂移方向，在四道 PMFUP 接近的菜品中实现了有效区分。

（3）**AMMBC 的辅助增强**：红烧牛腩饭的 AMMBC 得分（0.328）最高，表明与其行为相似的用户对该菜品偏好较强，协同过滤为高 PRMIDM 菜品提供了额外支撑，而未将低 PRMIDM 菜品错误提升——体现了收缩因子与置信度加权的保护作用。

融合排名前两名（65.7%与 65.6%）得分接近，形成有效竞争；与前两名后两名存在明显分差（约 27 个百分点），推荐排序区分度良好。

5.3.3 多餐前序惩罚效果

在 FUS6101 航班的两餐场景中，测试前序惩罚机制对推荐排序的影响。选取一名在午餐预选了"经典红烧牛腩饭"的用户，对比加餐推荐中惩罚前后的得分变化，结果如表 5.2 所示。

菜品	惩罚类型	惩罚前	惩罚后	排名变化
经典红烧牛腩饭	相同菜品 ×0.70	0.655	**0.459**	仍为第 2
川香麻辣牛肉面	同类型×0.85	0.657	**0.558**	仍为第 1
香烤鳕鱼时蔬饭	同类型×0.85	0.391	**0.332**	3→4
全素菌菇烩饭	无惩罚	0.384	**0.384**	4→2

由表 5.2 可见：与午餐相同的牛腩饭虽排名未变（2→2），但从惩罚前仅落后麻辣牛肉面 0.002 分（0.655 vs 0.657）扩大至落后 0.099 分（0.459 vs 0.558），与第一名彻底拉开差距；麻辣牛肉面虽被同类型惩罚降权仍保持首位；素食烩饭因不受惩罚从第 4 自然升至第 2，为旅客提供了与午餐不同的餐食选择。该机制在不移除任何菜品的前提下，通过差异化降权实现了"避免重复推荐"的设计目标。

5.3.4 舱位差异化推荐验证

舱位递进规则要求高舱位可见低舱位的所有餐食。测试结果验证：头等舱用户在 FUS6101 上可见 6 道菜品（4 道经济舱通用 + 2 道头等舱专属），商务舱与经济舱可见 4 道（此航线无商务舱专属绑定）。通过检查 SQL 查询日志确认筛选条件为`WHERE cabin_type IN (1,2,3)`（头等舱）、`IN (2,3)`（商务舱）和`IN (3)`（经济舱），与舱位递进规则一致。

5.4 系统功能验证

5.4.1 管理端功能验证

管理端功能验证采用逐页操作确认的方式，覆盖九个模块的核心业务流程。验证要点包括：登录认证与 JWT 令牌签发、航空总览看板四项统计指标（推荐总次数、评分均值、热门菜品分布、异常用户数）的正确展示、航班的增删改查与航线-菜品绑定操作、菜品的 mealType 与 flavorTags 字段配置、评分任务的状态筛选与重开操作、用户餐食的按航班需求统计。全部验证用例均通过。管理端航空总览看板的运行截图如图 5.1 所示。

> **[此处插入图 5.1：管理端航空总览看板截图]**

5.4.2 小程序端功能验证

小程序端验证覆盖了从微信登录到航后评分的完整用户闭环：微信授权登录获取OpenID→18 位身份证号核验与舱位展示→航班选择绑定→偏好设置（含跳过冷启动入口）→推荐列表浏览与菜品预选确认→预选改选→评分提交与暂缓。小程序端五步闭合的核心页面截图如图 5.2 至图 5.4 所示。

> **[此处插入图 5.2：小程序航班绑定页 + 偏好采集页截图（可合并排版）]**
> **[此处插入图 5.3：小程序餐食预选推荐页截图]**
> **[此处插入图 5.4：小程序预选确认页 + 航后评分页截图（可合并排版）]**

5.5 本章小结

本章从测试环境、数据集构建、算法效果分析和系统功能验证四个方面进行了全面测试。全部 29 个测试用例均通过。算法效果分析表明：融合权重自适应机制实现了冷启动到个性化的平滑过渡；三子模型协同互补，PMFUP 提供基线锚定、PRMIDM 实现漂移区分、AMMBC 提供邻居增强；前序惩罚正确将相同菜品降权 30%、同类型降权 15%；舱位递进规则在 SQL 层正确生效。管理端九大模块与小程序端五步闭环的全部功能验证均通过，系统运行稳定。

第 6 章 结论与展望

6.1 研究工作总结

本文针对航空餐食推荐场景中供给约束强、偏好动态变化和交互信号稀疏的特征，设计并实现了一个面向航空旅客的智能美食推荐系统，提出了 Fused-PMFUP-PRMIDM-AMMBC 三子模型融合的自适应混合推荐算法。主要工作包括：完成了航空餐食推荐场景的领域建模，涵盖航线-菜品绑定、舱位等级递进、多餐时序约束和库存限额等特殊约束，设计了 18 张数据表；提出了由 PMFUP（内容匹配）、PRMIDM（漂移检测）和 AMMBC（协同过滤）构成的自适应混合推荐算法，三子模型权重根据用户行为信号强度动态调整，并引入前序餐食惩罚机制（相同菜品×0.70、同类型×0.85）优化多餐推荐体验；基于 Spring Boot 3 + Vue3 + UniApp 技术栈构建了管理端、小程序端和后端服务三端系统，实现了偏好采集、智能推荐、预选确认、航后评分的完整闭环，共含 65 个 REST API 端点和两个定时任务；通过 90 名用户、10 道菜品、4 个航班的模拟数据集和 29 个测试用例进行了功能验证与算法效果分析，验证了三子模型的协同互补机制和系统的功能稳定性。

6.2 研究不足与展望

本文工作存在以下不足：（1）测试基于模拟数据，缺乏航空餐食领域的真实用户行为数据支撑；（2）基于用户的协同过滤在用户规模增大时计算复杂度较高；（3）超参数基于经验设定，缺乏系统化调优；（4）未引入航班时段、季节等上下文特征，偏好建模粒度有待细化；（5）缺乏精确率、召回率等离线评价指标的量化评估。未来工作可从以下方向展开：接入真实航空旅客数据进行算法验证与调优；引入近似最近邻搜索或基于物品的协同过滤优化计算效率；通过网格搜索或贝叶斯优化对超参数进行自动寻优；融合航班时段、航程时长等上下文特征提升偏好建模精度；建立包含离线指标计算和在线 A/B 实验的完整评价体系。

参考文献

[1] 余江龙，宋腾飞，汪德超，等. 个性化推荐系统研究方法综述[J]. 电脑知识与技术，2024，(10)：76-80.

[2] 王素芳，李明. 基于协同过滤和深度学习的混合推荐算法研究[J]. 信息技术与信息化，2022，(5)：45-49.

[3] 夏翔，张涛，刘威. 基于混合相似度和用户兴趣迁移的改进协同过滤推荐算法[J]. 计算机时代，2023，(3)：28-33.

[4] 毛骞，等. 推荐系统冷启动问题解决方法研究综述[J]. 计算机科学与探索，2024，18(5)：1197-1215.

[5] 刘超，张宇，陈敏. 基于辅助信息与长短期偏好的序列推荐[J]. 计算机应用研究，2024，41(1)：32-39.

[6] 钱聪，等. 基于用户多重兴趣漂移特征权重的网络出版物推荐研究[J]. 数据分析与知识发现，2023，(8).

[7] 刘昊东，王诚. 基于热门度修正因子和置信度的协同过滤算法[J]. 计算机技术与发展，2023，33(3)：127-132.

[8] 改进用户评分相似度的协同过滤推荐算法[J]. 现代计算机，2022，(21)：33-38.

[9] 陈南平. 融合情境感知特征的就餐推荐模型研究[J]. 信息与电脑(理论版)，2023，35(14)：48-51.

[10] 孟佩，党小娟，刘冬. 基于深度学习和多领域融合的个性化美食与旅游推荐系统[J]. 现代信息科技，2023，7(18)：115-119.

[11] 李凯. 基于项目特征与用户偏好的美食推荐系统的设计与实现[D]. 南昌航空大学，2025.

[12] 武森，董雅贤，魏桂英，等. 面向稀疏数据的协同过滤用户相似度计算研究[J]. 计算机科学与探索，2022，16(5)：1043-1052.

[13] 申艳梅，李亚平，王岩. 基于用户属性和项目属性的融合推荐算法[J]. 河南理工大学学报(自然科学版)，2022，41(2)：131-137.

[14] 杨硕，史亚平. 基于 Python+Flask 的在线考试系统设计与实现[J]. 电脑知识与技术，2025，21(02)：47-49+56.

[15] 谢鸿博. 前后端分离架构下基于图神经网络的社交网络关系挖掘[J]. 信息技术与信息化，2024，(12)：104-107.

[16] Koren Y, Bell R, Volinsky C. Matrix factorization techniques for recommender systems[J]. Computer, 2009, 42(8): 30-37.

[17] Bondevik J N, Bennin K E, Babur Ö, et al. A systematic review on food recommender systems[J]. Expert Systems with Applications, 2024, 238: 122166.

[18] Dwarampudi B B R. AI-driven personalization in inflight catering: From passenger profiles to plate[J]. Journal of Computational Science and Technology Studies, 2025, 7(8): 1-15.

[19] Herlocker J L, Konstan J A, Terveen L G, et al. Evaluating collaborative filtering recommender systems[J]. ACM Transactions on Information Systems (TOIS), 2004, 22(1): 5-53.

致　谢

南昌航空大学，感谢相遇，后会有期。